

Line-by-Line Mathematical Guide for HAETAE_Events.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Named rejection and acceptance events.

How To Read This Script

Read this file as event vocabulary. It gives stable names to predicates that later game hops should cite without unfolding.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore.</code>	Imports AllCore. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_Algebra.</code>	Imports HAETAE_Params HAETAE_Algebra. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
4	<code>theory HAETAE_Events.</code>	Begins the namespace HAETAE_Events, grouping the declarations as one checked theory.
5	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
6	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
7	<code>import HAETAE_Algebra.</code>	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
8	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
9	<code>op keygen_rejection_failure (md : mode) (pk : pkey) (sk : skey) : bool =</code>	Defines predicate keygen_rejection_failure, an event or well-formedness condition used in later implications and game proofs.
10	<code>! valid_keypair md pk sk /\ ! secret_norm_ok md sk.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
11	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
12	<code>op signing_rejection_failure (md : mode) (sig : signature) : bool =</code>	Defines predicate signing_rejection_failure, an event or well-formedness condition used in later implications and game proofs.
13	<code>! valid_signature md sig /\ ! response_norm_ok md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
14	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
15	<code>op verification_rejection_failure (md : mode) (sig : signature) : bool =</code>	Defines predicate <code>verification_rejection_failure</code> , an event or well-formedness condition used in later implications and game proofs.
16	<code>! valid_signature md sig \ / ! verify_norm_ok md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
17	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
18	<code>op any_rejection_failure (md : mode) (pk : pkey) (sk : skey)</code>	Defines operation <code>any_rejection_failure</code> , a total mathematical function or abbreviation used by later declarations.
19	<code>(sig : signature) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
20	<code>keygen_rejection_failure md pk sk \ /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
21	<code>signing_rejection_failure md sig \ /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
22	<code>verification_rejection_failure md sig.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
23	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
24	<code>lemma keygen_rejection_free_current md sd :</code>	States checked lemma <code>keygen_rejection_free_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
25	<code>! keygen_rejection_failure md (keygen_internal md sd).‘1</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	<code>(keygen_internal md sd).‘2.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
27	<code>proof.</code>	Begins the proof script for the preceding proposition.
28	<code>by rewrite /keygen_rejection_failure /valid_keypair /secret_norm_ok</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
29	<code>/secret_norm_sq /keygen_internal /secret_key_of_seed; case md.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
31	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
32	<code>lemma signing_rejection_free_current md sk m ctx coins :</code>	States checked lemma <code>signing_rejection_free_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
33	<code>! signing_rejection_failure md (sign_internal md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
34	<code>proof.</code>	Begins the proof script for the preceding proposition.
35	<code>rewrite /signing_rejection_failure.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
36	<code>rewrite valid_signature_sign_internal response_norm_ok_current.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
37	<code>by rewrite /=.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
38	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
39	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
40	<code>lemma verification_rejection_free_current md sk m ctx coins :</code>	States checked lemma <code>verification_rejection_free_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
41	<code>! verification_rejection_failure md (sign_internal md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	<code>proof.</code>	Begins the proof script for the preceding proposition.
43	<code>rewrite /verification_rejection_failure.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
44	<code>rewrite valid_signature_sign_internal verify_norm_ok_current.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
45	<code>by rewrite /=.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
46	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
47	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
48	<code>lemma accepted_transcript_rejection_free_current md sd m ctx coins :</code>	States checked lemma <code>accepted_transcript_rejection_free_current</code> . The following proof script must derive this proposition from prior definitions and lemmas.
49	<code>! any_rejection_failure md (keygen_internal md sd).‘1</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	<code>(keygen_internal md sd).‘2</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
51	<code>(sign_internal md (keygen_internal md sd). '2 m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
52	<code>proof.</code>	Begins the proof script for the preceding proposition.
53	<code>by rewrite /any_rejection_failure keygen_rejection_free_current</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
54	<code>signing_rejection_free_current verification_rejection_free_current.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
55	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
56	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
57	<code>end HAETAE_Events.</code>	Closes the current theory or module block.